message explaining what exactly a particular change will do.

For example, your commit message can say *'Add new hello.c file'*.

Tip: Make sure your commit messages are short (about 80 characters) and meaningful. If you have to write a long description, first write your short commit message and then press *Return* twice to go to the third line and start writing as long a description as you want.

Now save and exit the editor by pressing *Ctrl+O* and *Ctrl+X*.

After you are done making the changes, it's time to push them to the remote server for everyone to access.

Run the following command:

```
git push origin master
```

Git will ask you to enter your user name and password. Go ahead and do so. Once you are done, you will see that your files have been uploaded to GitHub with the new commit messages.

Now if you are working on the same repository with someone else, chances are that they have made some changes before you uploaded your changes.

To download all of their new changes, simply run the following command:

```
git pull origin master
```

This will re-synchronise your local repository with the copy on the server. This is the typical workflow while on a project. However, you may need to do a lot of other things sometimes like checking out a previous commit, or modifying the Git history.

To see the history of commits in your repository, run:

```
git log
```

This will list out all the changes made along with their commit hashes and commit messages.

If you want to go back to a previous commit but still have the files intact (which means, the new changes will be untracked), use the command given below:

```
git reset <commit-hash>
```

The *commit-hash* is the long string of characters after the word *commit* in the Git log. This can be used to uniquely identify a commit.

If you want to go back to a specific commit by undoing all the changes after it, you can run:

```
git reset --hard <commit-hash>
```

Warning: Use this feature sparingly – only when you are sure that you have no other choice but to lose your current changes.

Sometimes we just want to see the state of the files in a previous branch without having to lose all our future changes. So, to simply go back in time temporarily and come back, you can run:

```
git checkout <previous-commit>
```

To return to the present, run:

```
git checkout master
```

Sometimes we need to exclude some files from source control so that others can't see them – for example, configuration files containing sensitive information like database passwords and API keys, IDE configuration files like *.idea* for IntelliJ IDEA users, or auto-generated files that can be reproduced from the files present in the repository like binaries.

To do that, we create a special file called *.gitignore*. In this file, simply write the pattern of files or directories to exclude from the source control.

An example *.gitignore* file looks like what's shown below:

```
.idea/
binaries/
db-config.yml
*.exe
```

.idea and *binaries* are folders that will be excluded entirely. They will not be uploaded to the Git server. *db-config.yml* is a file containing our database configuration, and any file matching that name will be excluded as well. **.exe* tells Git to exclude all the files that end with *.exe*. Here, '*' is a wild card character, which means that any set of characters can take its place.

This *.gitignore* file also has to be added to the repository by calling *git add .gitignore*, committing and pushing it to the server.

Warning: Do not add sensitive information like passwords and API keys to source control. Always add them to *.gitignore*. As a good practice, add a *<config-filename>.example* to the repository with fake values which other collaborators can rename with their own keys and passwords.

In this tutorial we have looked at the basic steps to get you started with Git and GitHub on Windows and to get you familiar with the version control system. Git is a very powerful tool and can be used in many ways. Covering all the aspects of it is beyond the scope of this tutorial.

I recommend that you go through *https://git-scm.com/docs/gittutorial* and start using Git for your next project so that you learn more about it. END

By: Dedipyaman Das

The author is a software developer and open source enthusiast.